



NAMA : KARTIKA TRI JULIANA
PROGRAM STUDI : D4 – Sistem Informasi Bisnis
KELAS : 2B
NIM : 2341760116

JOBSHEET 10

RESTFUL API

Sebelumnya kita sudah membahas mengenai *authentication*, *authorization*, dan *middleware* pada Laravel. Dimana kita telah membuat fungsi login, register, logout, serta pemilihan role dan penerapan session pada halaman web. Pada pertemuan kali ini, kita akan mempelajari penerapan RESTFUL API di dalam project Laravel.

Sebelum kita masuk materi, kita buat dulu project baru yang akan kita gunakan untuk membangun aplikasi sederhana dengan topik *Point of Sales (PoS)*, sesuai dengan **Studi Kasus PWL.pdf**.
Jadi kita bikin project Laravel 10 dengan nama **PWL_POS**.

Project PWL_POS akan kita gunakan sampai pertemuan 12 nanti, sebagai project yang akan kita pelajari

A. RESTFUL API

Representational State Transfer (REST) adalah gaya arsitektur perangkat lunak yang mendefinisikan seperangkat prinsip untuk merancang jaringan aplikasi terdistribusi. RESTful API adalah aplikasi pemrograman antarmuka yang mengikuti prinsip-prinsip REST untuk mentransfer data antara klien dan server.

RESTful API adalah salah satu arsitektur dalam API (*Application Program Interface*) yang menggunakan request HTTP untuk mengakses data. Data diakses dengan menggunakan HTTP method GET, PUT, POST dan DELETE yang merujuk pada operasi pembacaan, pembaruan, pembuatan dan penghapusan pada resource. Selain HTTP method, dalam RESTful atau REST digunakan juga HTTP response untuk mendefinisikan respon data yang dikembalikan. Format respon yang umum digunakan berupa JSON (Javascript Object Notation).



B. JSON Web Token (JWT)

JWT adalah singkatan dari JSON Web Token. Ini adalah standar terbuka (RFC 7519) yang mendefinisikan format token yang kompak dan mandiri untuk mentransfer klaim antara dua pihak. JWT sering digunakan dalam otentikasi dan pertukaran informasi yang aman di lingkungan yang tidak terpercaya, seperti internet.

JWT terdiri dari tiga bagian yang dipisahkan oleh titik ("."): header, payload, dan signature. Setiap bagian ini terdiri dari data JSON yang dienkripsi menggunakan algoritma tertentu dan kemudian disatukan untuk membentuk token yang lengkap. Header berisi jenis token dan tipe algoritma yang digunakan untuk enkripsi. Payload berisi klaim atau informasi yang ingin disampaikan. Signature digunakan untuk memverifikasi bahwa token belum berubah dan datanya berasal dari sumber yang dipercayai.

JWT sering digunakan dalam sistem otentikasi dan otorisasi modern, seperti aplikasi web dan layanan web API, karena fleksibilitasnya dalam menyampaikan informasi terenkripsi secara ringkas.

Kita dapat menggunakan JWT untuk:

- Authentication

Ketika pengguna melakukan authentication dan mendapatkan token, maka setiap permintaan berikutnya akan menyertakan token tersebut, dan memungkinkan pengguna untuk mengakses route, service, dan resources yang diizinkan.

- Pertukaran informasi

JSON Web Token adalah cara yang baik untuk mengirimkan informasi antar pihak dengan aman. Dengan token yang sudah ditandatangani dengan algoritma RSA, maka kita bisa tahu siapa yang melakukan request tersebut.

Berikut adalah cara kerja JWT :

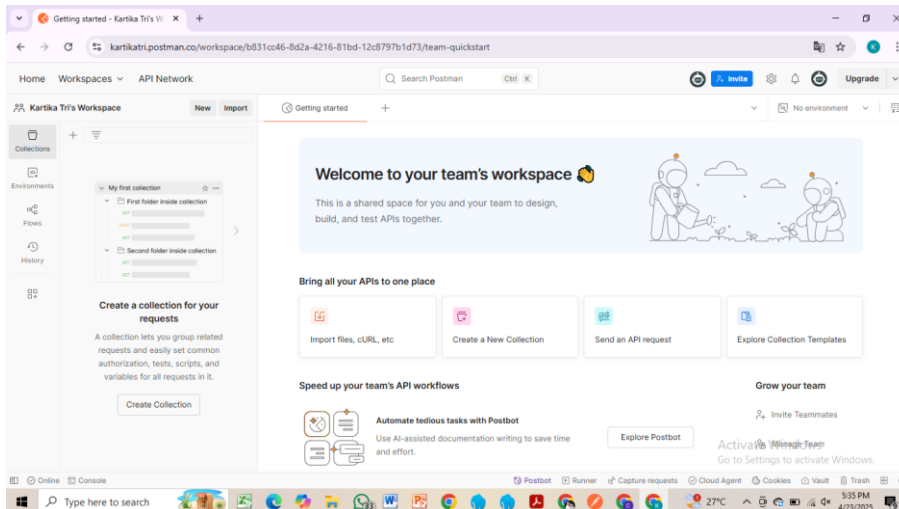
JWT (JSON Web Token) adalah cara untuk mentransfer informasi antara dua pihak secara aman sebagai objek JSON. Ini terdiri dari tiga bagian: header, payload, dan signature. Setelah pengguna berhasil autentikasi, server menghasilkan token JWT yang disematkan dalam permintaan HTTP. Server kemudian memvalidasi token untuk memberikan akses ke sumber daya yang diminta. Ini memberikan autentikasi yang aman dan stateless tanpa memerlukan penyimpanan status sesi di server.



Praktikum 1 – Membuat RESTful API Register

1. Sebelum memulai membuat REST API, terlebih dahulu download aplikasi Postman di <https://www.postman.com/downloads>.

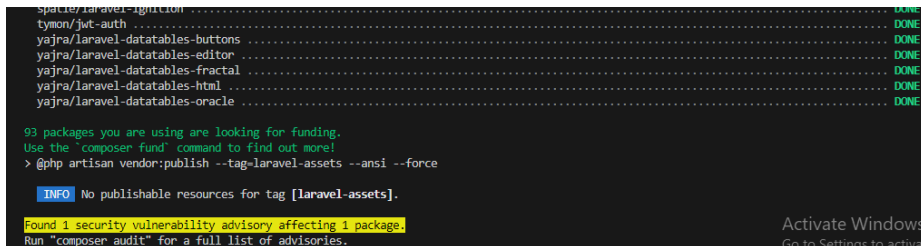
Aplikasi ini akan digunakan untuk mengerjakan semua tahap praktikum pada Jobsheet ini.



2. Lakukan instalasi JWT dengan mengetikkan perintah berikut:

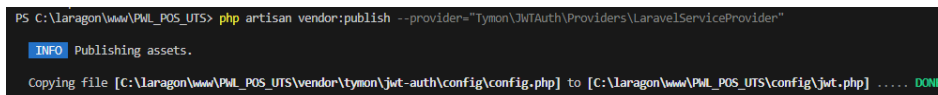
```
composer require tyson/jwt-auth:2.1.1
```

Pastikan Anda terkoneksi dengan internet.



3. Setelah berhasil menginstall JWT, lanjutkan dengan publish konfigurasi file dengan perintah berikut:

```
php artisan vendor:publish --  
provider="Tyson\JWTAuth\Providers\LaravelServiceProvider"
```



4. Jika perintah di atas berhasil, maka kita akan mendapatkan 1 file baru yaitu config/jwt.php. Pada file ini dapat dilakukan konfigurasi jika memang diperlukan.
5. Setelah itu jalankan perintah berikut untuk membuat secret key JWT.

```
php artisan jwt:secret
```

Jika berhasil, maka pada file .env akan ditambahkan sebuah baris berisi nilai key JWT_SECRET.



```
PS C:\laragon\www\PWL_POS_UTS> php artisan jwt:secret  
jwt-auth secret [yHcvMh95F33CegejrbEeAyCBfI4qJsN2btHNZDKKYdEKv2dID2sKsfRC19kjgIk] set successfully.  
PS C:\laragon\www\PWL_POS_UTS>
```

6. Selanjutnya lakukan konfigurasi guard API. Buka config/auth.php. Ubah bagian 'guards' menjadi seperti berikut.

```
'guards' => [  
    'web' => [  
        'driver' => 'session',  
        'provider' => 'users',  
    ],  
    'api' => [  
        'driver' => 'jwt',  
        'provider' => 'users',  
    ],  
],
```

7. Kita akan menambahkan kode di model UserModel, ubah kode seperti berikut:

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Model;  
use Tymon\JWTAuth\Contracts\JWTSubject;  
use Illuminate\Foundation\Auth\User as Authenticatable;  
  
class UserModel extends Authenticatable implements JWTSubject  
{  
    public function getJWTIdentifier(){  
        return $this->getKey();  
    }  
  
    public function getJWTCustomClaims(){  
        return [];  
    }  
  
    protected $table = 'm_user';  
    protected $primaryKey = 'user_id';  
}
```



8. Berikutnya kita akan membuat controller untuk register dengan menjalankan perintah berikut.

```
php artisan make:controller Api/RegisterController
```

Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama RegisterController.

9. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Models\UserModel;
6  use App\Http\Controllers\Controller;
7  use Illuminate\Http\Request;
8  use Illuminate\Support\Facades\Validator;
9
10 class RegisterController extends Controller
11 {
12     public function __invoke(Request $request)
13     {
```



```
14 //set validation
15 $validator = Validator::make($request->all(), [
16     'username' => 'required',
17     'nama' => 'required',
18     'password' => 'required|min:5|confirmed',
19     'level_id' => 'required'
20 ]);
21
22 //if validations fails
23 if($validator->fails()){
24     return response()->json($validator->errors(), 422);
25 }
26
27 //create user
28 $user = UserModel::create([
29     'username' => $request->username,
30     'nama' => $request->nama,
31     'password' => bcrypt($request->password),
32     'level_id' => $request->level_id,
33 ]);
34
35 //return response JSON user is created
36 if($user){
37     return response()->json([
38         'success' => true,
39         'user' => $user,
40     ], 201);
41 }
42
43 //return JSON process insert failed
44 return response()->json([
45     'success' => false,
46 ], 409);
47 }
48 }
```

10. Selanjutnya buka routes/api.php, ubah semua kode menjadi seperti berikut.



```
<?php

use App\Http\Controllers\Api\RegisterController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

/*
|-----
| API Routes
|-----
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
```

11. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman.

Postman interface showing a POST request to `http://localhost/PWL_POS_UTS/public/api/register`. The response is a JSON object with validation errors:

```
{
  "username": [
    "The username field is required."
  ],
  "nama": [
    "The nama field is required."
  ],
  "password": [

```

Buka aplikasi Postman, isi URL `localhost/PWL_POS/public/api/register` serta method POST. Klik Send.

Jika berhasil akan muncul error validasi seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.

12. Sekarang kita coba masukkan data. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data registrasi menggunakan nilai yang Anda inginkan.



Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

Key	Value
password	pwjls10api
password_confirmation	pwjls10api
level_id	2

Body Cookies (1) Headers (9) Test Results Status: 201 Created Time: 1192 ms Size: 468 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": true,
3   "user": {
4     "username": "Tikataka15",
5     "nama": "Kartika Tri Juliana",
6     "level_id": "2",
7     "updated_at": "2025-04-23T12:31:22.000000Z",
8     "created_at": "2025-04-23T12:31:22.000000Z",
```

Setelah klik tombol Send, jika berhasil maka akan keluar pesan sukses seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.

13. Lakukan commit perubahan file pada Github.

Praktikum 2 – Membuat RESTful API Login

1. Kita buat file controller dengan nama LoginController.

`php artisan make:controller Api/LoginController`

Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama LoginController.

2. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```
1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use App\Http\Controllers\Controller;
6 use Illuminate\Http\Request;
7 use Illuminate\Support\Facades\Validator;
8
```




```
9  class LoginController extends Controller
10 {
11     public function __invoke(Request $request)
12     {
13         //set validation
14         $validator = Validator::make($request->all(), [
15             'username' => 'required',
16             'password' => 'required'
17         ]);
18
19         //if validation fails
20         if ($validator->fails()) {
21             return response()->json($validator->errors(), 422);
22         }
23
24         //get credentials from request
25         $credentials = $request->only('username', 'password');
26
27         //if auth failed
28         if (!$token = auth()->guard('api')->attempt($credentials)) {
29             return response()->json([
30                 'success' => false,
31                 'message' => 'Username atau Password Anda salah'
32             ], 401);
33         }
34
35         //if auth success
36         return response()->json([
37             'success' => true,
38             'user' => auth()->guard('api')->user(),
39             'token' => $token
40         ], 200);
41     }
42 }
```

3. Berikutnya tambahkan route baru pada file api.php yaitu /login dan /user.

```
use App\Http\Controllers\Api\LoginController;

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
Route::post('/login', App\Http\Controllers\Api\LoginController::class)->name('login');
Route::middleware('auth:api')->get('/user', function (Request $request) {
    return $request->user();
});
```

4. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/login serta method POST. Klik Send.



http://localhost/PWL_POS_UTS/public/api/register

POST http://127.0.0.1:8000/api/login

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

Key	Value
Key	Value

Body Cookies (1) Headers (9) Test Results

Status: 422 Unprocessable Content Time: 378 ms Size: 385 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "username": [
3     "The username field is required."
4   ],
5   "password": [
6     "The password field is required."
7   ]
8 }
```

Jika berhasil akan muncul error validasi seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.

5. Selanjutnya, isikan username dan password sesuai dengan data user yang ada pada database. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data user. Klik tombol Send, jika berhasil maka akan keluar tampilan seperti berikut. Copy nilai token yang diperoleh pada saat login karena akan diperlukan pada saat logout.

http://localhost/PWL_POS_UTS/public/api/register

POST http://127.0.0.1:8000/api/login

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

Key	Value
☒ username	Tikataka15
☒ password	pwlljs10api
Key	Value

Body Cookies (1) Headers (9) Test Results

Status: 200 OK Time: 920 ms Size: 801 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": true,
3   "user": {
4     "user_id": 43,
5     "level_id": 2,
6     "foto": null,
7     "username": "Tikataka15",
8     "nama": "Kartika Tri Juliana",
9   }
10 }
```



Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.

6. Lakukan percobaan yang untuk data yang salah dan berikan screenshoot hasil percobaan Anda.
7. Coba kembali melakukan login dengan data yang benar. Sekarang mari kita coba menampilkan data user yang sedang login menggunakan URL `localhost/PWL_POS/public/api/user` dan method GET. **Jelaskan hasil dari percobaan tersebut.**

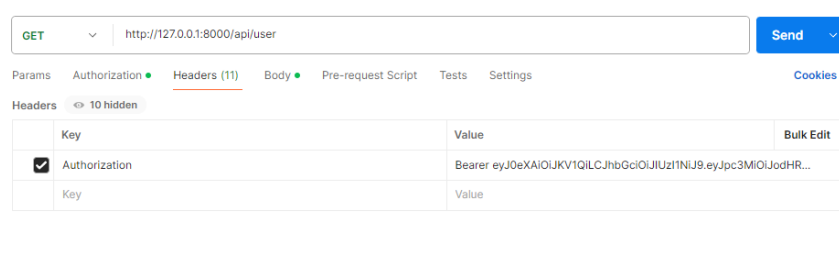
- Copy url token setelah melakukan login

```
9      "created_at": "2025-04-23T12:31:22.000000Z",
10      "updated_at": "2025-04-23T12:31:22.000000Z"
11    },
12    "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ3c3M1OjJodHRwOi8vMTI3LjAuMC4xOjgwMDAvYXBPb2xvZ2luIiwiaWF0IjoxNzQ1NDU0OTc3LCJleHAiOjE3NDU0MTg1NzcsIm5iZiI6MTc0NTQxNDk3NywiYW50IjoizHFDdF1XY2ZwZlA2bU9CayIsInN1YiI6IjQzIiwicHJ2Ijo1NDkzZjg4MzRmMmI1ODY3MGMvYTYwYVYwYVZlZGVmNDIzNDEzNzAwNjkwYyY39.1vG_s_tMqf0xT3FvQPatp8VP9_FwGBsI85QC6uHVxSw"
13  }
```

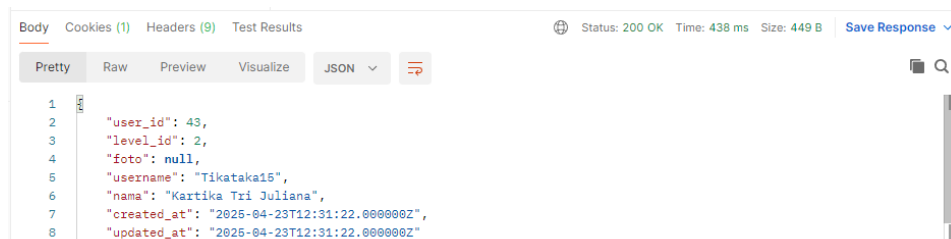
- Mengubah URL menjadi user kemudian ganti POST ke GET

GET `http://127.0.0.1:8000/api/user`

- Tambahkan url token di value



Hasil :



8. Lakukan commit perubahan file pada Github.



Praktikum 3 – Membuat RESTful API Logout

1. Tambahkan kode berikut pada file .env

`JWT_SHOW_BLACKLIST_EXCEPTION=true`

```
JWT_SECRET=yHcvMh95F33CegejrbEeAyCBfI4qJsN2bthNZDKKYdEKv2dID2sKsfRC19kjjgIk
JWT_SHOW_BLACKLIST_EXCEPTION=true
```

2. Buat Controller baru dengan nama LogoutController.
`php artisan make:controller Api/LogoutController`
3. Buka file tersebut dan ubah kode menjadi seperti berikut.

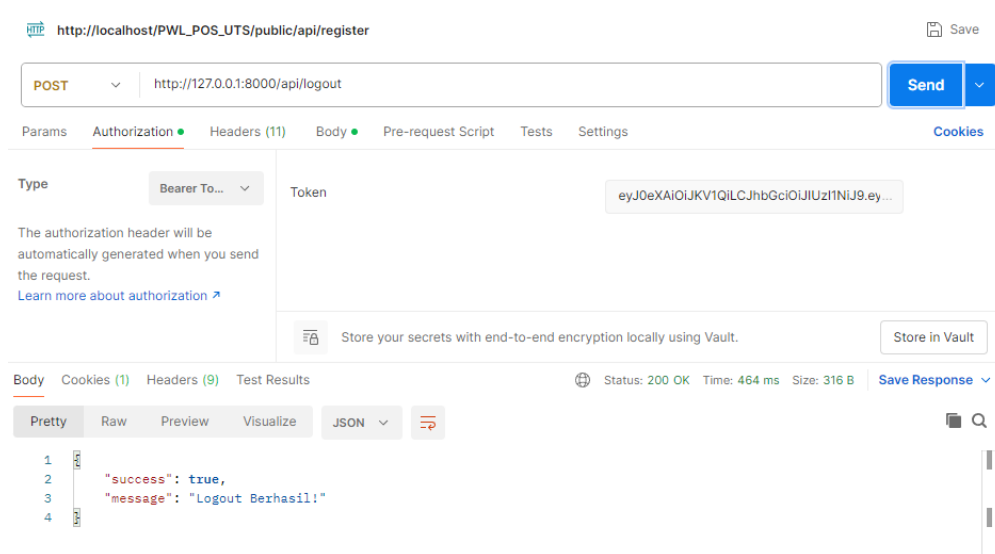
```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4  use Illuminate\Http\Request;
5  use App\Http\Controllers\Controller;
6  use Tymon\JWTAuth\Facades\JWTAuth;
7  use Tymon\JWTAuth\Exceptions\JWTException;
8  use Tymon\JWTAuth\Exceptions\TokenExpiredException;
9  use Tymon\JWTAuth\Exceptions\TokenInvalidException;
10
11 class LogoutController extends Controller
12 {
13     public function __invoke(Request $request)
14     {
15         //remove token
16         $removeToken = JWTAuth::invalidate(JWTAuth::getToken());
17
18         if($removeToken) {
19             //return response JSON
20             return response()->json([
21                 'success' => true,
22                 'message' => 'Logout Berhasil!',
23             ]);
24         }
25     }
26 }
```



4. Lalu kita tambahkan routes pada api.php

```
Route::post('/logout', App\Http\Controllers\Api\LogoutController::class)->name('logout');
```

5. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/logout serta method POST.
6. Isi token pada tab Authorization, pilih Type yaitu Bearer Token. Isikan token yang didapat saat login. Jika sudah klik Send.



Lakukan percobaan yang sama dan berikan screenshot hasil percobaan Anda.

7. Lakukan commit perubahan file pada Github.

Praktikum 4 – Implementasi CRUD dalam RESTful API

Pada praktikum ini kita akan menggunakan tabel m_level untuk dimodifikasi menggunakan RESTful API.

1. Pertama, buat controller untuk mengolah API pada data level.
`php artisan make:controller Api/LevelController`
2. Setelah berhasil, buka file tersebut dan tuliskan kode seperti berikut yang berisi fungsi CRUDnya.

```
namespace App\Http\Controllers\Api;  
use App\Http\Controllers\Controller;  
use Illuminate\Http\Request;  
use App\Models\LevelModel;  
  
class LevelController extends Controller  
{  
    public function index()  
    {  
        return LevelModel::all();  
    }  
}
```



```
public function store(Request $request)
{
    $level = LevelModel::create($request->all());
    return response()->json($level, 201);
}

public function show(LevelModel $level)
{
    return LevelModel::find($level);
}

public function update(Request $request, LevelModel $level)
{
    $level->update($request->all());
    return LevelModel::find($level);
}

public function destroy(LevelModel $user)
{
    $user->delete();

    return response()->json([
        'success' => true,
        'message' => 'Data terhapus',
    ]);
}
```

3. Kemudian kita lengkapi routes pada api.php.

```
use App\Http\Controllers\Api\LevelController;

Route::get('levels', [LevelController::class, 'index']);
Route::post('levels', [LevelController::class, 'store']);
Route::get('levels/{level}', [LevelController::class, 'show']);
Route::put('levels/{level}', [LevelController::class, 'update']);
Route::delete('levels/{level}', [LevelController::class, 'destroy']);
```

4. Jika sudah. Lakukan uji coba API mulai dari fungsi untuk menampilkan data. Gunakan URL: localhost/PWL_POS-main/public/api/levels dan method GET. **Jelaskan dan berikan screenshot hasil percobaan Anda.**

= hasil yang didapat dari praktikum 4 adalah akan memunculkan data level yang sesuai di database



```
Body Cookies (1) Headers (9) Test Results Status: 200 OK Time: 423 ms Size: 1.17 KB Save Response
Pretty Raw Preview Visualize JSON
1 [
2   {
3     "level_id": 1,
4     "level_kode": "CEO",
5     "level_nama": "CEO",
6     "created_at": "2025-04-07T11:21:10.000000Z",
7     "updated_at": null
8   },
9   {
10    "level_id": 2,
11    "level_kode": "SLS",
12    "level_nama": "Sales",
13    "created_at": "2025-04-07T11:21:10.000000Z",
14    "updated_at": null
15  },
16  {
17    "level_id": 3,
18    "level_kode": "DRT",
19    "level_nama": "Direktur",
```

5. Kemudian, lakukan percobaan penambahan data dengan URL : localhost/PWL_POS-main/public/api/levels dan method POST seperti di bawah ini.

```
POST localhost/PWL_POS/public/api/levels Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
Key Value Description Bulk Edit
level_kode Text SPV
level_nama Text Supervisor
Key Value Description
Body Cookies Headers (11) Test Results Status: 201 Created Time: 276 ms Size: 531 B Save as example
Pretty Raw Preview Visualize JSON
1 {
2   "level_kode": "SPV",
3   "level_nama": "Supervisor",
4   "updated_at": "2024-04-22T21:48:32.000000Z",
5   "created_at": "2024-04-22T21:48:32.000000Z",
6   "level_id": 4
7 }
```

Jelaskan dan berikan screenshoot hasil percobaan And



6. Berikutnya lakukan percobaan menampilkan detail data. **Jelaskan dan berikan screenshot hasil percobaan Anda.**

= dengan menambahkan key level_nama dan level_kode saja dan tidak perlu level_id karena akan otomatis terisi sesuai urutan yang ada

none form-data x-www-form-urlencoded raw binary

Key	Value	...	Bulk Edit
☒ level_kode	PWS		
☒ level_nama	Pengawas		
Key	Value		

Body Cookies (1) Headers (9) Test Results Status: 201 Created Time: 445 ms Size: 420 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "level_kode": "PWS",
3   "level_nama": "Pengawas",
4   "updated_at": "2025-04-23T14:02:40.000000Z",
5   "created_at": "2025-04-23T14:02:40.000000Z",
6   "level_id": 10
7 }
```

7. Jika sudah, kita coba untuk melakukan edit data menggunakan localhost/PWL_POS-main/public/api/levels/{id} dan method PUT. Isikan data yang ingin diubah pada tab Param.

PUT localhost/PWL_POS-main/public/api/levels/4?level_kode=SPR Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
☒ level_kode	SPR			
Key	Value	Description		

Body Cookies Headers (11) Test Results Status: 200 OK Time: 266 ms Size: 528 B Save as example

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "level_id": 4,
4     "level_kode": "SPR",
5     "level_nama": "Supervisor",
6     "created_at": "2024-04-22T21:40:32.000000Z",
7     "updated_at": "2024-04-22T21:48:19.000000Z"
8   }
9 ]
```

Jelaskan dan berikan screenshot hasil percobaan Anda.

= menggunakan PUT untuk update data yang dicari, karena data yang dicari adalah kode levelnya 4 dengan level_nama nya adalah PWS maka yang muncul hanya data PWS saja



http://localhost/PWL_POS_UTS/public/api/register

PUT http://127.0.0.1:8000/api/levels/10?level_kode=PWS

Params Authorization Headers (11) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

Key	Value	...	Bulk Edit
☒ level_kode	PWS		
☒ level_nama	Pengawas		
Key	Value		

Body Cookies (1) Headers (9) Test Results

Status: 200 OK Time: 420 ms Size: 417 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "level_id": 10,
3   "level_kode": "PWS",
4   "level_nama": "Pengawas",
5   "created_at": "2025-04-23T14:02:40.000000Z",
6   "updated_at": "2025-04-23T14:02:40.000000Z"
7 }
8
```

8. Terakhir lakukan percobaan hapus data. **Jelaskan dan berikan screenshoot hasil percobaan Anda.**
9. Lakukan commit perubahan file pada Github.



TUGAS

Implementasikan CRUD API pada tabel lainnya yaitu tabel m_user, m_kategori, dan m_barang

1. M_user

a. Create

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/users`. The request body is in form-data with the following fields:

Key	Value
username	Kartika15
nama	Kartika Tri Juliana
password	12345

The response status is 201 Created, with a time of 1240 ms and a size of 455 B. The response body is in JSON format:

```
1 {
2   "level_id": "1",
3   "foto": null,
4   "username": "Kartika15",
5   "nama": "Kartika Tri Juliana",
6   "updated_at": "2025-04-24T06:24:34.000000Z",
7   "created_at": "2025-04-24T06:24:34.000000Z",
8   "user_id": 46
9 }
```

b. Read

The screenshot shows a Postman interface for a GET request to `http://127.0.0.1:8000/api/kategoris`. The response status is 200 OK, with a time of 428 ms and a size of 1.24 KB. The response body is in JSON format:

```
1 {
2   "user_id": 37,
3   "level_id": 3,
4   "foto": null,
5   "username": "chris",
6   "nama": "chan",
7   "created_at": "2025-04-09T09:44:22.000000Z",
8   "updated_at": "2025-04-09T09:44:22.000000Z"
9 },
10 {
11   "user_id": 39,
```



c. Update

The screenshot shows a Postman interface for a PUT request to `http://127.0.0.1:8000/api/users`. The request body is a JSON object with the following fields: `user_id` (39), `level_id` (6), `foto` (null), `username` (Admin), `nama` (Kartika), `created_at` (2025-04-14T09:58:05.000000Z), and `updated_at` (2025-04-14T09:58:05.000000Z). The response status is 200 OK.

```
PUT http://127.0.0.1:8000/api/users
{
  "user_id": 39,
  "level_id": 6,
  "foto": null,
  "username": "Admin",
  "nama": "Kartika",
  "created_at": "2025-04-14T09:58:05.000000Z",
  "updated_at": "2025-04-14T09:58:05.000000Z"
}
```

Status: 200 OK Time: 660 ms Size: 432 B

d. Delete

The screenshot shows a Postman interface for a DELETE request to `http://127.0.0.1:8000/api/users/42?nama=hana`. The request body is a JSON object with the following fields: `success` (true), and `message` (Data user terhapus). The response status is 200 OK.

```
DELETE http://127.0.0.1:8000/api/users/42?nama=hana
{
  "success": true,
  "message": "Data user terhapus"
}
```

Status: 200 OK Time: 376 ms Size: 318 B

2. M_kategori

a. Create

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/kategoris`. The request body is a JSON object with the following fields: `kategori_id` (7), `kategori_kode` (ATK017), and `kategori_nama` (Alat Tulis Kantor). The response status is 201 Created.

```
POST http://127.0.0.1:8000/api/kategoris
{
  "kategori_kode": "ATK017",
  "kategori_nama": "Alat Tulis Kantor",
  "updated_at": "2025-04-24T06:27:35.000000Z",
  "created_at": "2025-04-24T06:27:35.000000Z",
  "kategori_id": 8
}
```

Status: 201 Created Time: 651 ms Size: 440 B



b. Read

GET http://127.0.0.1:8000/api/kategoris

Status: 200 OK Time: 581 ms Size: 1.25 KB

```
1 {
2   "kategori_id": 1,
3   "kategori_kode": "111",
4   "kategori_nama": "sweater",
5   "created_at": "2025-03-24T06:47:32.000000Z",
6   "updated_at": "2025-03-24T06:47:32.000000Z",
7 },
8 {
9   "kategori_id": 2,
10  "kategori_kode": "121",
11  "kategori_nama": "kaos",
12  "created_at": "2025-03-24T06:47:45.000000Z",
13  "updated_at": "2025-03-24T06:47:45.000000Z",
14 }
```

c. Update

PUT http://127.0.0.1:8000/api/kategoris/1?kategori_nama=sweater

Status: 200 OK Time: 537 ms Size: 422 B

```
1 {
2   "kategori_id": 1,
3   "kategori_kode": "111",
4   "kategori_nama": "sweater",
5   "created_at": "2025-03-24T06:47:32.000000Z",
6   "updated_at": "2025-03-24T06:47:32.000000Z",
7 }
```

d. Delete

DELETE http://127.0.0.1:8000/api/kategoris/7?kategori_nama=sepatu

Status: 200 OK Time: 408 ms Size: 310 B

```
1 {
2   "message": "Kategori berhasil dihapus",
3 }
```



3. M_barang

a. Create

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/barangs`. The request body is in form-data with the following fields:

Field	Value
barang_kode	ATK017
barang_nama	Pensil
harga_beli	1500
harga_jual	2500

The response is shown in JSON format:

```
{  "kategori_id": "8",  "barang_kode": "ATK017",  "barang_nama": "Pensil",  "harga_beli": "1500",  "harga_jual": "2500",  "updated_at": "2025-04-24T06:34:47.000000Z",  "created_at": "2025-04-24T06:34:47.000000Z",  "barang_id": 32}
```

b. Read

The screenshot shows a Postman interface for a GET request to `http://127.0.0.1:8000/api/barangs`. The response is shown in JSON format:

```
{  "barang_id": 4,  "kategori_id": 1,  "barang_kode": "122",  "barang_nama": "Baju",  "harga_beli": 30000,  "harga_jual": 50000,  "created_at": "2025-03-24T06:48:23.000000Z",  "updated_at": "2025-03-24T06:48:23.000000Z"}
```

c. Update

The screenshot shows a Postman interface for a PUT request to `http://127.0.0.1:8000/api/barangs/23?barang_kode=BRG003`. The request body is in form-data with the following fields:

Field	Value
barang_kode	BRG003

The response is shown in JSON format:

```
{  "barang_id": 23,  "kategori_id": 2,  "barang_kode": "BRG003",  "barang_nama": "Barang 3",  "harga_beli": 26863,  "harga_jual": 84257,  "created_at": null,  "updated_at": null}
```



d. Delete

DELETE

Params • Authorization • Headers (8) Body • Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ barang_kode	BRG003	
Key	Value	

Body Cookies Headers (9) Test Results

Pretty Raw Preview Visualize JSON

```
1 [
2   "success": true,
3   "message": "Data terhapus"
4 ]
```

*** Sekian, dan selamat belajar ***